An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary retains comparisons in directed acyclic graphs and represents possible orders by a region of real positions satisfying interval bounds and the recorded inequalities. It estimates the fraction of this region's volume retained by each answer through deterministic calculations on bounded graph neighborhoods. Moving items into ordered subproblems controls graph degrees and directed path lengths; a geometric analysis shows that these moves preserve the optimal asymptotic bound.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

Deterministic adversaries from graph entropy. Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These

accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir, and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the earlier adversaries. Section 3 gives our data structure and comparison procedure. Items are stored at nodes of a rooted tree, with a directed acyclic graph recording comparisons within each node. Interval bounds and the graph's inequalities define a region of real positions representing the possible orders there. The adversary estimates the fraction of this region's volume retained by each answer. Three technical ingredients make this possible.

Deterministic estimates from bounded neighborhoods. After propagating interval bounds through the full graph, we compute the required probability exactly in a bounded neighborhood. To prove accuracy, we compare uniform assignments to the full region and to this smaller region. Updating corresponding positions with the same random choices makes the queried coordinates approach one another; the degree and path-length bounds control how far these updates can transmit a difference (Section 6).

Choosing positions with large remaining volume. Moving an item into a child fixes its position in the parent. A geometric inequality turns estimates of the probability on either side of a position into a lower bound on the volume left after fixing it. This supplies the guarantee needed by the position search (Section 5.3).

Bounding the cost of graph simplification. Section 4 bounds the number of refinements and the sum of their anchor depths over the whole run. Counting removed arcs and tracking changes in volume give two inequalities controlling these totals. A probability argument then relates the final state to the single total order determined by the sorter.

Finally, Section 7 lets the degree and path-length bounds grow slowly with n, keeping the computation polynomial as the comparison coefficient approaches one.

Appendix A proves the density estimates, and Appendix B collects the parameters. The manuscript source is available online.[1]

2 Related Work

2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

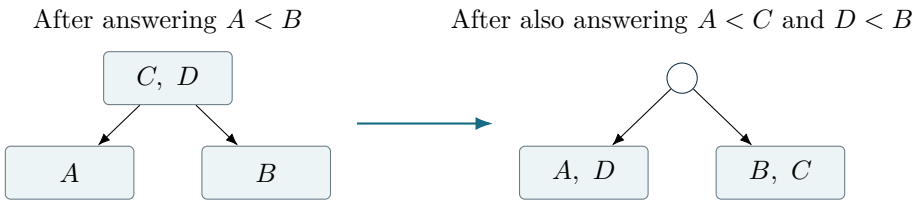


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree

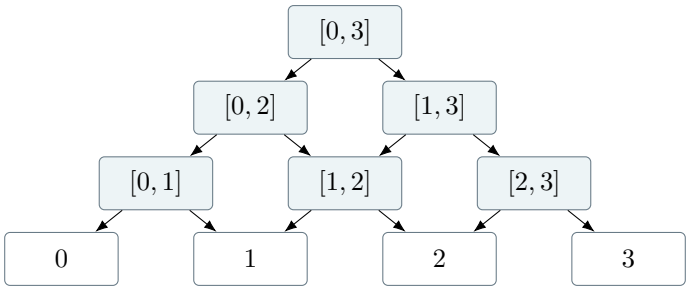


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

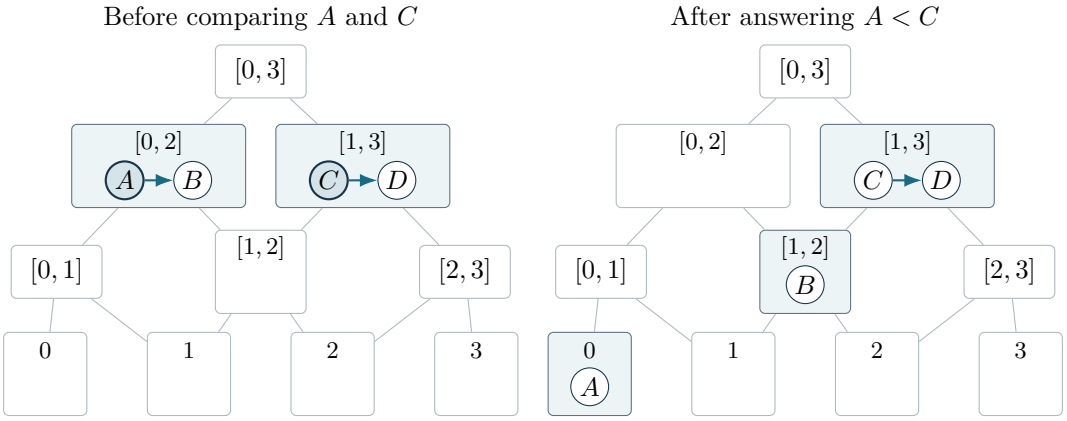


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

3 The adversary

We describe the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [6] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of b and c undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in that node's DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. We call this its *in-node DAG*. Figure 4 shows an example.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields a_x, b_x restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. The bounds a_x, b_x are fractions whose denominators are powers of two; we call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In an in-node DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. Each answered comparison between free items contributes at most one arc.

A *feasible assignment* at a node is a choice of real values for its position variables satisfying $a_x < X_x < b_x$ for every item x and $X_x < X_y$ for every arc $x \to y$ in its in-node DAG.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables X_x are unassigned, and the root's in-node DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

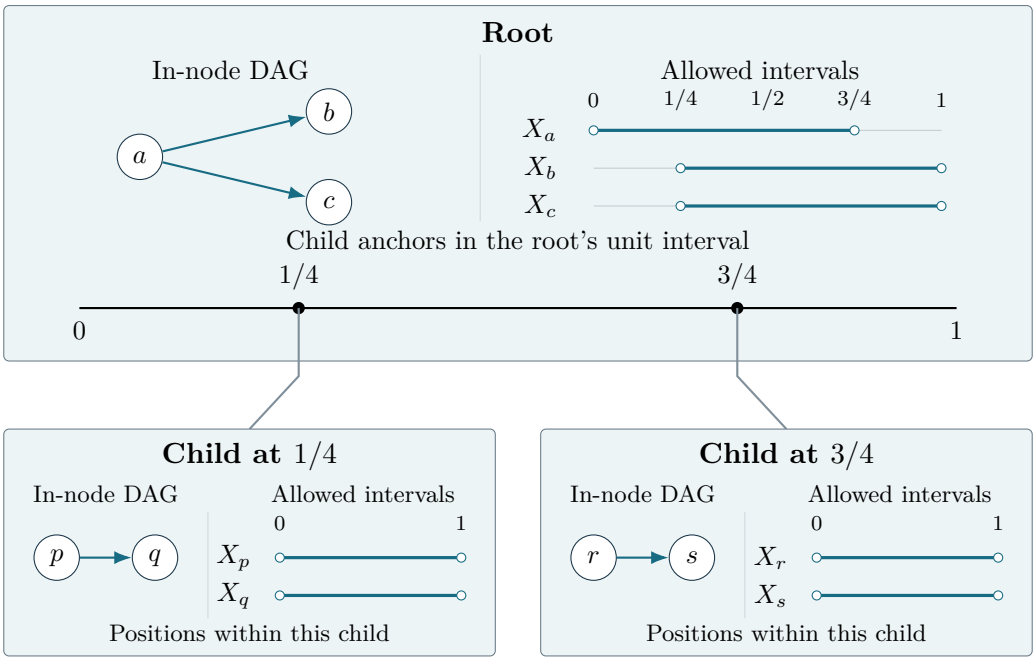


Figure 4: A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 4, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. These answers form the *global transcript*, an output stream of answers to the sorter. We retain their order constraints in a *global comparison DAG* on the original item labels. When a comparison starts with incomparable labels in this DAG, its answer $A < B$ adds the arc $A \to B$. An in-node DAG contains the comparisons between items currently stored at that node. Initially the global transcript is empty, and the global comparison DAG has all n labels as vertices and no arcs.

3.2 Answering a comparison

Algorithm 2 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare A and B, first return any answer already implied by a directed path in the global comparison DAG. Otherwise, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors determine the answer immediately. In the other cases, first propagate the interval bounds through the in-node DAG. If there is an arc $y \to x$, every lower bound on X_y is also a lower bound on X_x; upper bounds propagate in the reverse direction. Write $y \preceq x$ when $y = x$ or a directed path leads from y to x. The resulting *effective interval* is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{1}$$

The operation `Tighten` (Algorithm 1) stores these bounds as a_x, b_x. It propagates lower bounds in topological order and upper bounds in reverse topological order. It is performed after every change to its in-node DAG or intervals. Bounds remain stored even if the arcs that implied them are later removed.

Algorithm 1: `Tighten`(v): propagate interval bounds through an in-node DAG.

Input: A tree node v, its in-node DAG, and the stored interval bounds.
Output: The stored bounds $a_x = \ell_x$ and $b_x = r_x$ of (1) for every item x at v.
$T \leftarrow$ a topological order of the in-node DAG at v, choosing by item label whenever several vertices are eligible
foreach x *in* T **do**
  $a_x \leftarrow \max(\{a_x\} \cup \{a_y : y \to x\})$
foreach x *in reverse order of* T **do**
  $b_x \leftarrow \min(\{b_x\} \cup \{b_y : x \to y\})$

An interval bound or a directed path may already decide the comparison; in that case return the determined answer.

For an undecided comparison, let p be the probability that A precedes B when positions at this node are assigned uniformly subject to its interval bounds and in-node DAG inequalities. The subroutine `Estimate` returns a deterministic estimate $\widehat{p}$. Fix a rational error tolerance $0 < \eta < 1/4$. Choose $A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise. For two free items, the event being estimated is $X_A < X_B$. For a free item x and an anchor c, estimate $X_x < c$, taking the complement if the queried order is reversed. An answer between two free items adds an arc to the in-node DAG; an answer involving an anchor adds an interval bound. Repeated comparisons never add duplicate arcs.

The estimator also requires a lower bound on the width of at least one queried free coordinate. Write this threshold as $1/R$, with rational $R \geq 1$. Call a coordinate *wide* when $W_x \geq 1/R$, and *narrow* otherwise. If all queried free coordinates are narrow, move one of their items into a child before trying the query again. Such a move is called a *refinement*, denoted `Refine`(x); Section 3.3 gives the operation in Algorithm 4.

The calculation also needs bounds on the in-node DAG. Fix positive integers k and H. We limit each vertex to fewer than k neighbors, counting both incoming and outgoing arcs, and each directed path to fewer than H vertices. After answering, refine vertices with at least k neighbors until none remain. Then select a path of H vertices and refine all of them, repeating until no such path remains. The procedure `Repair`(v) performs these *repairs* at node v; see Algorithm 3. A path repair includes the entire selected list, even if earlier moves break the path.

The following lemma supplies the accuracy needed by the comparison rule and the anchor search. Its degree and path bounds also cover intermediate states during a repair. Section 6 gives the estimator and proves the lemma.

Lemma 3.1 (Probability estimation)**.** *Suppose a node admits a feasible assignment, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. Under the uniform distribution on these assignments,* `Estimate` *deterministically computes, with error at most η:*

(i) $\mathbb{P}(X_x < c)$*, when x is wide and c is dyadic;*
(ii) $\mathbb{P}(X_x < X_y)$*, when at least one of x, y is wide.*

Algorithm 2: `Compare`(A, B): answer a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
if *a directed path in the global comparison DAG determines the answer* **then**
  answer ← the determined answer
  Append *answer* to the output transcript
  return *answer*
while *true* **do**
  v ← the root
  while *A and B lie in the same child subtree of v* **do**
    v ← that child
  if *A and B lie in different child subtrees of v* **then**
    answer ← the left-to-right order of their child anchors
    Append *answer* to the output transcript
    Insert its arc into the global comparison DAG
    return *answer*
  `Tighten`(v)
  if *interval bounds or a directed path in the in-node DAG at v determine the answer* **then**
    answer ← the determined answer
    Append *answer* to the output transcript
    Insert its arc into the global comparison DAG
    return *answer*
  if *at least one queried free coordinate is wide* **then**
    break
  `Refine`(the selected queried free item)
$\widehat{p}$ ← `Estimate`$(v, A < B, \eta)$
answer ← $A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise
Append *answer* to the output transcript
Insert its arc into the global comparison DAG
Store *answer* as an arc in the in-node DAG at v or as an interval bound
`Tighten`(v)
`Repair`(v)
return *answer*

Refining a queried item restarts the search from the root because the node that handles the query may change. Both `Compare` and `Repair` choose the smallest label among eligible items. `Repair` selects the lexicographically first eligible path and processes its items in label order. These rules make every selection deterministic.

Algorithm 3: `Repair`(v): restore the degree and path-length bounds.

Input: A tree node v and the current adversary state.
Output: The updated state, with degree below k and paths of fewer than H vertices in the in-node DAG at v.

```
while the in-node DAG at v has a vertex of degree at least k do
    Refine(such a vertex)
while the in-node DAG at v has a directed path on H vertices do
    P ← the selected path
    foreach x ∈ P in label order do
        Refine(x)
```

3.3 Moving an item into a child

Algorithm 4 gives `Refine`(x), which moves x from its current node into a child. After tightening the parent's interval bounds, it calls `ChooseAnchor` to select a position c inside x's effective interval.

We select anchor positions using a fixed infinite binary tree, the *implicit anchor tree*, shown in Figure 5. The same tree is used at every node of the adversary's tree. Each anchor-tree node u has an interval $I(u) = [a, b]$ and a position $c(u) = (a + b)/2$; write $|I(u)| = b - a$. The root has interval $[0, 1]$; the left and right children of u have intervals $[a, c(u)]$ and $[c(u), b]$, respectively.

At depth m, the 2^m nodes have intervals of length 2^{-m} and positions

$$c = (i + \tfrac{1}{2})2^{-m}, \qquad i = 0, \ldots, 2^m - 1. \tag{2}$$

We call m the *anchor depth*, always measured in the anchor tree. These positions are dyadic rationals, so refinement keeps the interval endpoints a_x, b_x dyadic. A finite path from the anchor tree's root specifies an anchor. Algorithm 5 in Section 5.3 gives the walks that select an anchor inside the effective interval.

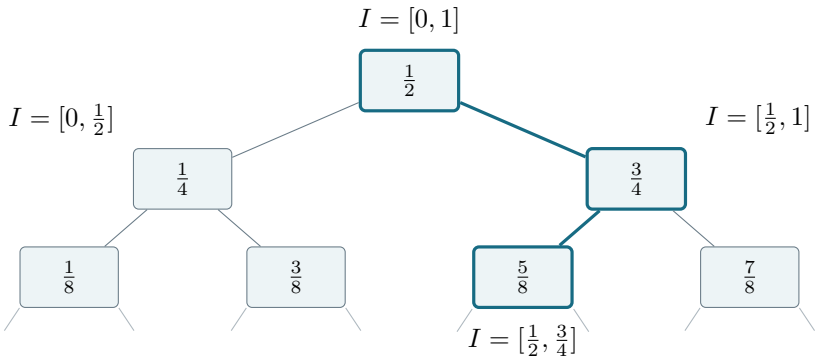


Figure 5: The implicit anchor tree. The highlighted path selects 5/8 at depth 2.

Fixing $X_x = c$ places x in the child block anchored at c. The arcs incident to x in that in-node DAG become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, retain the bounds, and propagate them through the remaining in-node DAG. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 6 illustrates reuse of an existing child.

Algorithm 4: `Refine`(x): move an item into a child.

Input: A free item x and the current adversary state.
Output: The updated state, with x stored in a child of its former node.
$v \leftarrow$ the node storing x
`Tighten`(v)
$c \leftarrow$ `ChooseAnchor`(x)
foreach *arc* $y \to x$ *in the in-node DAG at* v **do**
  $b_y \leftarrow \min(b_y, c)$
foreach *arc* $x \to z$ *in the in-node DAG at* v **do**
  $a_z \leftarrow \max(a_z, c)$
Remove x's record from v and its vertex and incident arcs from the in-node DAG at v
`Tighten`(v)
if v *has no child anchored at* c **then**
  Create a child of v at c, with no records and an empty in-node DAG
$u \leftarrow$ the child of v anchored at c
Insert a record $(x, X_x, 0, 1)$ at u, with a fresh, unassigned position variable X_x
Insert x as an isolated vertex in the in-node DAG at u
`Tighten`(u)

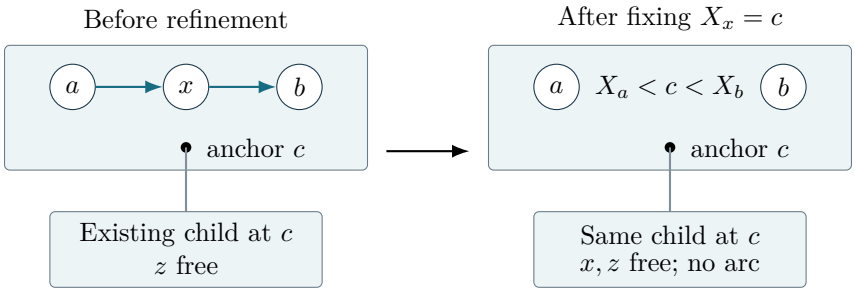


Figure 6: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.2 proves that this operation preserves all answered comparisons.

Refinement can impose additional order relations. For example, suppose the root contains A, B, C with only $A < B$ recorded. Refining A at $1/2$ restricts B to $(1/2, 1)$. Refining C into the same child then leaves both $A < C < B$ and $C < A < B$ possible, while fixing the additional relation $C < B$. Section 4 shows that we can afford this loss of possible orders while retaining the optimal asymptotic comparison bound.

Table 1 collects the thresholds used by Algorithms 2, 3, and 4 above. In the next section, we explain why the adversary forces any correct sorter to perform many comparisons.

4 Why the adversary forces many comparisons

Recall from Algorithm 3 that a larger degree threshold k lets the adversary retain more comparisons involving each item before moving it into a child. Increasing k lets the adversary force a comparison count closer to $n \log n$. The following bound quantifies this dependence.

Theorem 4.1 (Comparison bound)**.** *For every power of two $k \geq 64$, the adversary of Algorithm 2, with the remaining parameter choices in Table 1, is deterministic and legal, and every call terminates. Every correct sorter makes at least*

$$\frac{n \log n - 3n}{1 + (2 \log k + 18)/k} \tag{3}$$

comparisons.

For this section, fix a power of two $k \geq 64$. Section 7 chooses k as a function of n so that it grows without bound while the adversary's running time remains polynomial.

To measure the cost of refinement, we count how often it occurs and sum the chosen anchor depths over the whole interaction. We will bound both totals in terms of the number of comparisons. A final probability argument connects these bounds to the $n!$ possible orders of the items.

4.1 Volume changes and anchor selection

We next introduce the definitions required to state Lemma 4.2 and prove Theorem 4.1.

To assess an anchor choice c for x, we compare the volume left for the other items after fixing $X_x = c$ with the node's original feasible volume. Their ratio is the density $f_x(c)$. Let v be the node containing x; its feasible assignments form the region

$$K_v = \{X : a_x < X_x < b_x \text{ for every free } x, \quad X_x < X_y \text{ for every in-node arc } x \to y\}. \tag{4}$$

Write $V_v = \mathrm{vol}(K_v)$, and let $S_x(c)$ be the volume of the section obtained by fixing $X_x = c$, measured in the remaining coordinates. Under the uniform distribution on K_v, the density of X_x is

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\, dt.$$

To track volume across the adversary's tree, take the product of the node regions. Each item has a position variable at exactly one node, so this product has dimension n and volume

$$Z = \prod_v V_v. \tag{5}$$

A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \leq 1$.

Choose the estimation error to be

$$\eta = \frac{1}{100k}. \tag{6}$$

By Lemma 3.1, each estimated answer retains at least $1/2 - \eta$ of its node's volume. Its decrease in $\log Z$ is therefore at most

$$a_\eta = -\log(1/2 - \eta) \leq 1 + \frac{1}{k}. \tag{7}$$

For the inequality, use $2^{-1/k} \leq 1 - 1/(2k) \leq 1 - 1/(50k) = 1 - 2\eta$. An answer already determined by the stored constraints leaves Z unchanged.

For refinement we need a lower bound on $f_x(c)$ in terms of the anchor's depth in the implicit anchor tree. Put

$$B = (k+1)^{2H+1}. \tag{8}$$

In an in-node DAG of degree at most k and directed paths of at most $2H$ vertices, every coordinate comparable with x lies within undirected distance $2H$ of x. Thus B bounds the number of other coordinates comparable with x.

Lemma 4.2 (Anchor selection)**.** *Suppose a node's feasible region has positive volume, its interval endpoints are dyadic, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. For $R = 2^{2kB}$ and rational $0 < \eta < 1/32$,* `ChooseAnchor` *deterministically returns an anchor $c \in (\ell_x, r_x)$ at depth m in the implicit anchor tree satisfying*

$$\log f_x(c) \geq (1 - 1/k)m - 4. \tag{9}$$

If x is narrow, then

$$m > 2kB + 2. \tag{10}$$

Section 5.3.3 proves Lemma 4.2 and explains the choice of R. The density bound controls the change in Z through the following identity. Refinement fixes X_x in the parent and gives x a fresh unit coordinate in the child.

Lemma 4.3 (Refinement identity)**.** *Suppose every node region has positive volume. Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{11}$$

Section 5.2 proves this identity, including when the child already contains items.

To apply the probability and anchor estimates throughout a call, we need positive node volumes and bounds on the in-node DAGs. We also need each call to return an answer consistent with the earlier answers.

Proposition 4.4 (Consistency and termination)**.** *With the parameter choices of Theorem 4.1, every reached state has positive node volumes and represents only orders consistent with the global transcript. Every comparison call terminates and performs at most $n + 2$ refinements. During a call, every in-node DAG has degree at most k and directed paths of at most $2H$ vertices. At the end of the call, the degrees are below k and the directed paths have fewer than H vertices.*

Section 5.4 proves Proposition 4.4 from the comparison and refinement rules, using Lemmas 3.1 and 4.2.

4.2 Counting refinements

After q completed comparison calls, let P be the number of refinements and D the sum of their anchor depths. We first bound P using the arcs removed by degree repairs and the depths used by the other refinements.

Each degree refinement removes at least k in-node arcs. At most one such arc is inserted per comparison, and each inserted arc is removed at most once. Thus there are at most q/k degree refinements. A preparation refinement uses a narrow coordinate, so its anchor depth exceeds $2k$ by Lemma 4.2.

For path repairs, choose

$$H = 2^k. \tag{12}$$

The items on a selected path must enter distinct children. Their anchors therefore occupy distinct nodes of the implicit anchor tree. Since there are few nodes at small depths, a batch of H such refinements has large total anchor depth.

Lemma 4.5 (Sum of anchor depths along a path)**.** *The H labels of a selected path in an in-node DAG enter distinct children of their parent in the adversary's tree. If their anchors have depths $m_1, \ldots, m_H$ in the implicit anchor tree, then*

$$\sum_{i=1}^{H} m_i \geq H(k - 2). \tag{13}$$

Section 5.5 proves the distinct-child claim and counts the available anchors. This is why Algorithm 3 finishes the whole selected path even when earlier refinements have broken it.

Lemma 4.6 (Refinement count)**.** *After q completed calls,*

$$P \le \frac{q}{k} + \frac{D}{k-2}. \tag{14}$$

Proof. There are at most q/k degree refinements. Every preparation refinement has depth greater than $k-2$, and every complete path batch has average depth at least $k-2$. The number of these remaining refinements is therefore at most $D/(k-2)$. □

4.3 Bounding the total anchor depth

Refinement increases $\log Z$ by at least $(1-1/k)m-4$ when its anchor has depth m, by Lemmas 4.2 and 4.3. Starting from $Z = 1$ and adding the changes over q completed calls gives

$$\log Z \ge -a_\eta q + (1-1/k)D - 4P. \tag{15}$$

But $Z \le 1$. Hence the volume gained by deep refinements is limited by the volume removed by comparisons:

$$(1-1/k)D \le a_\eta q + 4P. \tag{16}$$

Together with the refinement count, this bounds D in terms of q.

Lemma 4.7 (Total anchor depth)**.** *After q completed calls,*

$$D \le \frac{(k+5)(k-2)}{k^2-7k+2}\, q. \tag{17}$$

Consequently $D \le (1+O(1/k))q$ and $P = O(q/k)$.

Proof. Substitute (14) into (16) and use $a_\eta \le 1 + 1/k$:

$$\left(1 - \frac{1}{k} - \frac{4}{k-2}\right) D \le \left(1 + \frac{5}{k}\right) q.$$

The coefficient on the left is positive for $k \ge 64$. Rearranging gives (17); substituting it back into (14) gives the bound on P. □

4.4 The state at termination

The product Z measures freedom within individual nodes. For example, two items stored alone in separate children can each retain the full interval $(0,1)$, giving $Z = 1$, even though the child anchors fix their order. To account for the order determined by the tree, we use a random experiment on a fixed state.

Start at the root. At a visited node, each existing child whose anchor has depth m is chosen with probability

$$w(m) = \frac{2^{-(1+1/k)m}}{4k}. \tag{18}$$

Choosing a child continues the experiment there. With the remaining probability, stop and choose a uniform coordinate in $(0,1)$ at that node. The probabilities of the children sum to at most

$$\frac{1}{4k} \sum_{m \ge 0} 2^m 2^{-(1+1/k)m} = \frac{1}{4k(1-2^{-1/k})} \le \frac{1}{2}, \tag{19}$$

so the stopping probability is at least $1/2$. Each state has a finite tree, so the experiment produces a node and a coordinate.

Sample one such position independently for each of the n labels. The tree orders these positions by the same rule as in Section 3: compare a coordinate with the child anchors, and order positions inside one child recursively. Ties have probability zero, and all labels use the same distribution. Permuting the labels therefore shows that each relative order has probability $1/n!$.

Lemma 4.8 (Bound at termination)**.** *If the transcript determines a unique total order after q completed calls, then*

$$\log(n!) \le n - \log Z + (1 + 1/k)D + (\log k + 2)P. \tag{20}$$

Proof. Consider the event that every label stops at the node where its record is stored, and that the sampled coordinates satisfy all the in-node constraints. Conditional on the chosen stopping nodes, the coordinates are independent and uniform, so the probability of satisfying the constraints is Z.

The stopping probabilities contribute at least 2^{-n}. Since records only move downward, their required paths, counted separately for each label, contain exactly the P edge traversals made by refinements, with total anchor depth D. The child choices therefore contribute $2^{-(1+1/k)D-(\log k+2)P}$. Thus the event has probability at least

$$Z\, 2^{-n-(1+1/k)D-(\log k+2)P}.$$

By Proposition 4.4, the event produces an order consistent with the transcript. The unique such order has probability $1/n!$ in the random experiment. Comparing these probabilities and taking logarithms proves (20). □

4.5 The comparison bound

Proof of Theorem 4.1. Proposition 4.4 gives consistency and termination of every call. The procedures and their tie rules are deterministic. Suppose the sorter finishes after q comparisons. Substitute the volume bound (15) into (20):

$$\log(n!) \le n + a_\eta q + \frac{2D}{k} + (\log k + 6)P. \tag{21}$$

The two depth terms have nearly cancelled. Lemmas 4.6 and 4.7 now give the desired coefficient.

For the explicit bound, write $j = \log k$. Using $a_\eta \le 1 + 1/k$ and substituting (14) and (17) into (21) gives

$$\log(n!) \le n + \left(1 + \frac{2j + 18}{k}\right) q. \tag{22}$$

The coefficient inequality reduces, after multiplying by $k(k^2 - 7k + 2)$, to

$$3k^2 - (12j + 113)k + 2j + 42 \ge 0.$$

Indeed, $3 \cdot 2^j \ge 12j + 113$ for $j \ge 6$: the difference is positive at $j = 6$ and increases thereafter. Finally,

$$\log(n!) \ge \int_1^n \log x\, dx = n \log n - \frac{n}{\ln 2} + \frac{1}{\ln 2}.$$

Combining this estimate with (22) and $1 + 1/\ln 2 < 3$ proves (3). □

The next section proves the refinement and consistency guarantees used above.

5 Refinement and its guarantees

We now justify the refinement claims used in Section 4. First, any interior anchor preserves consistency with previous answers. Next we give `ChooseAnchor` and prove Lemma 4.2 using the probability estimates from Lemma 3.1. We then establish the in-node DAG bounds, legality of the full procedure, and the distinct-anchor property used to count path refinements.

5.1 Preserving the represented orders

We use the interpretation of the state given in Section 3. Assignments with equal free coordinates, or with a free coordinate equal to an anchor, lie on a finite union of sets of volume zero and are excluded when reading an order. We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

The global comparison DAG records every answered relation. The invariant must continue to hold when refinement removes an arc from an in-node DAG.

Lemma 5.1 (Effective intervals and sections)**.** *For an in-node DAG, tightening leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. Moreover, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

Proof. If $y \neq x$ precedes x on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds introduced by (1) already hold, so tightening changes no feasible assignment.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \leq \ell_y$ and $r_x \leq r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Increasing θ_x to θ_y strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints ℓ_y, r_y cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds c at its in-neighbors and lower bounds c at its out-neighbors. Upper bounds propagate only backward along arcs, and lower bounds only forward. Since all previous effective bounds remain stored, the new effective intervals are therefore

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the in-node DAG is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining in-node DAG, shows that the section has positive volume. □

Lemma 5.2 (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining x at an interior anchor c preserves both properties, including when the child at c already exists. The global transcript implies no relation between x and any label already in that child subtree.*

Proof. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the global comparison DAG.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

5.2 The volume gained by refinement

Recall that $Z = \prod_v V_v$ and that $f_x(c) = S_x(c)/V_v$ is the density of X_x under the uniform distribution on its node's feasible region. Fixing $X_x = c$ removes one coordinate from the parent; placing x in a child adds an independent unit coordinate there.

Proof of Lemma 4.3. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0, 1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0, 1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

For example, the parent region $0 < X_A < X_B < 1$ has area $1/2$. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length $3/4$. The new child coordinate $U_A \in (0, 1)$ turns this into a rectangle of area $3/4$, so refinement multiplies Z by $3/2$; see Figure 7. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

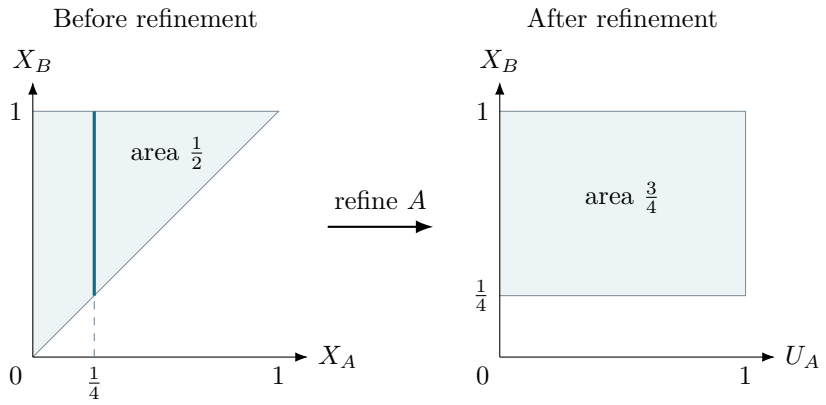


Figure 7: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

5.3 Choosing an anchor

Consider a positive-volume node whose in-node DAG has degree at most k and directed paths of at most $2H$ vertices. Algorithm 5 gives `ChooseAnchor`. Lemma 4.2 requires a lower bound on the density at the chosen position. `ChooseAnchor` starts at the root of the anchor tree. For a wide coordinate, we decide which child to visit by estimating the cumulative probability

$$F_x(t) = \mathbb{P}(X_x < t)$$

at the current anchor. For a narrow coordinate, we walk toward the centre of its effective interval until the anchor-tree interval is sufficiently short. Each probability estimate uses Lemma 3.1.

Algorithm 5: `ChooseAnchor`(x): choose a position for refinement.

Input: A free item x stored at node v, its effective interval (ℓ_x, r_x), and the fixed parameters R, η.
Output: A dyadic anchor $c \in (\ell_x, r_x)$.

```
W_x ← r_x − ℓ_x
u ← the root of the anchor tree
if W_x < 1/R then
    t ← (ℓ_x + r_x)/2
    while |I(u)| > W_x/4 do
        if t ≤ c(u) then
            u ← the left child of u
        else
            u ← the right child of u
    return c(u)
while true do
    p̂ ← Estimate(v, X_x < c(u), η)
    if p̂ < 1/4 then
        u ← the right child of u
    else if p̂ > 3/4 then
        u ← the left child of u
    else
        return c(u)
```

Each step increases the anchor depth by one and halves $|I(u)|$. The narrow branch follows the

child interval containing t, taking the left child when $t = c(u)$. This fixes its tie rule.

5.3.1 Density estimates used by the two rules

Suppose x has effective interval (ℓ, r) of width W and at most B other comparable labels. Then

$$f_x(t) \le \frac{B+1}{W}, \qquad f_x(t) \ge \frac{4^{-B}}{W} \quad \left(\ell + \frac{W}{4} \le t \le r - \frac{W}{4}\right). \tag{23}$$

The lower bound gives an endpoint-only choice: every point in the central half has this much density. The upper bound limits the probability in a short interval; it will bound how far the wide-item search descends in the anchor tree.

For the search we also need a density bound that improves as its search interval shrinks. For any $a \le c \le b$,

$$(b-a) f_x(c) \ge \min\{F_x(c), 1 - F_x(c)\}(F_x(b) - F_x(a)). \tag{24}$$

If both tails at c and the mass of $[a, b]$ are bounded below, this forces the density at c to be a constant times $1/(b-a)$. Appendix A proves (23) and (24) by explicit maps between sections.

5.3.2 The search for a wide item

Every test uses the same feasible region K and distribution F_x. Each test calls `Estimate` for $X_x < c$ with the same tightened endpoints. Only the queried threshold changes.

Lemma 5.3 (Wide-item search)**.** *The anchor-tree search for a wide item terminates at an interior anchor c. If the selected node has depth m, then*

$$f_x(c) \ge 2^{m-4}.$$

The search makes at most $\lceil \log(4(B+1)/W_x) \rceil + 1$ tests.

Proof. At the current node u of depth m, write $I(u) = [a, b]$ and $c = c(u)$. Every estimate has error at most $\eta < 1/32$. A test reported below $1/4$ has true value below $9/32$; the walk moves right, making c the new left endpoint. A test reported above $3/4$ has true value above $23/32$; the walk moves left, making c the new right endpoint. Since the initial endpoint values are $F_x(0) = 0$ and $F_x(1) = 1$, the search maintains

$$F_x(a) < 9/32, \qquad F_x(b) > 23/32.$$

At acceptance, the report lies in $[1/4, 3/4]$, so

$$\frac{7}{32} \le F_x(c) \le \frac{25}{32}, \qquad F_x(b) - F_x(a) \ge \frac{7}{16}.$$

The first bounds put c strictly inside its support. The node's interval has length 2^{-m}, so (24) gives

$$2^{-m} f_x(c) \ge \frac{7}{32}\frac{7}{16} = \frac{49}{512} > \frac{1}{16},$$

which proves the density guarantee.

For termination, the upper density bound in (23) gives, at every anchor-tree node reached,

$$\frac{7}{16} \le F_x(b) - F_x(a) \le \frac{B+1}{W_x} 2^{-m}.$$

At $m \ge \lceil \log(4(B+1)/W_x) \rceil$, the upper bound would be at most $1/4 < 7/16$. Starting at the root and descending one edge at each rejection therefore permits at most the stated number of tests. □

5.3.3 The anchor guarantee

For an anchor at depth m, the wide-item search gives $\log f_x(c) \geq m - 4$. For a narrow item, the central density estimate gives the weaker bound $m - 3 - 2B$. To absorb this extra loss into the allowance m/k, choose

$$R = 2^{2kB}. \tag{25}$$

The narrow walk stops when its node's interval first has length at most $W_x/4$. Thus $2^{-m} \leq W_x/4 < 1/(4R)$, forcing $m > 2kB + 2$. In particular $2B < m/k$, so both anchor rules satisfy the required density bound.

Proof of Lemma 4.2. The in-node DAG bounds imply that x has at most B other comparable coordinates, by (8). For a wide item, Lemma 5.3 gives the stronger lower bound $m - 4$. For a narrow item, let $b = 2^{-m}$ be the length of the selected node's interval. The stopping rule gives $W_x/8 < b \leq W_x/4$. That interval contains $t = (\ell_x + r_x)/2$, so its midpoint c is at distance at most $b/2 \leq W_x/8$ from t. It belongs to the central half and is therefore interior. By (23) and $W_x < 8b$,

$$f_x(c) \geq \frac{4^{-B}}{W_x} > 2^{m-3-2B}.$$

Moreover, $b \leq W_x/4 < 1/(4R)$, so $m > \log R + 2 = 2kB + 2$. In particular $2B < m/k$, and hence $m - 3 - 2B > (1 - 1/k)m - 4$, proving (9). The wide-item search terminates by Lemma 5.3. The narrow walk terminates because $W_x > 0$ and each step halves the interval length. Both walks are deterministic. □

The narrow walk continues to its prescribed depth even when an earlier anchor lies inside the effective interval. The resulting lower bound on m allows the weaker central density estimate to satisfy the same bound as the wide-item search.

5.4 Bounds during a comparison call

We next check how the in-node DAGs evolve. The first two lemmas bound the changes during a call. We then combine them with Lemmas 3.1 and 4.2 to prove that every call preserves feasibility and consistency with the answered comparisons.

Lemma 5.4 (Query preparation)**.** *Before selecting its answer,* Compare *performs at most two refinements. Each selected coordinate is narrow, and preparation preserves the in-node DAG bounds.*

Proof. A preparation refines one of the two original queried items. Its new coordinate is isolated and has interval $(0, 1)$. Before an answer is stored, any further preparation only removes a parent coordinate and adds an isolated coordinate in a child. It cannot restrict this fresh coordinate. If the query descends into that child, the prepared item is therefore still wide. Each queried item can require preparation at most once. The selection test ensures narrowness. Deleting vertices and admitting isolated vertices preserve the degree and path-length bounds. □

Lemma 5.5 (In-node DAG bounds and termination)**.** *Suppose every in-node DAG has degree less than k and directed paths of fewer than H vertices at the start of a call. If its selected answer leaves positive volume, then every in-node DAG has degree at most k and directed paths of at most $2H$ vertices throughout the call. At its end, every in-node DAG again satisfies the strict bounds.* `Repair` *performs at most n refinements, all at the node changed by the answer.*

Proof. Preparation preserves the strict bounds by Lemma 5.4. The answer adds at most one arc to the affected in-node DAG, raising any degree there by at most one. Any new directed path

consists of two old paths joined by that arc, so it has fewer than $2H$ vertices. A positive-volume answer cannot create a directed cycle. An interval change adds no in-node arcs.

Every refinement removes a vertex and its incident arcs from the parent. Thus degrees and path lengths cannot increase during restoration. The first loop of Algorithm 3 removes every degree violation. The second loop removes every path violation without increasing degrees. All selected path vertices are processed, even if earlier refinements have broken the original path. Each refinement removes a different free item from this node, so there are at most n. A child receives an isolated coordinate and keeps all its old arcs. Its degree and path-length bounds are preserved as well, so no child requires restoration. □

Consequently a complete call makes at most $n+2$ refinements. During the call, every coordinate has at most B other comparable coordinates in its in-node DAG: all lie within undirected distance $2H$ in an in-node DAG of degree at most k. These are the bounds used by the local estimator and the anchor rules.

Proof of Proposition 4.4. We induct on comparison calls and, within a call, on refinements. Initially the root region is the unit cube, every order is represented, and its in-node DAG has no arcs. Suppose the claims hold at the start of a call.

If the global comparison DAG already determines the answer, returning it preserves the represented orders and leaves the stored state unchanged. Otherwise the call proceeds with the recursive state.

Preparation refines only narrow coordinates. Lemma 4.2 supplies an interior position, so Lemma 5.2 preserves positivity and the order invariant. Lemma 5.4 shows that preparation terminates with the width needed for a probability query, unless anchors, interval bounds, or a directed path already decide it.

Lemma 3.1 now applies to the positive queried region with the strict in-node DAG bounds. Its chosen answer retains at least $1/2 - \eta > 0$ of the volume and makes that comparison true in every represented order. In particular, a new arc cannot create a directed cycle. An answer determined by anchors, interval bounds, or a directed path was already true in every represented order and leaves the represented orders unchanged. The selected answer is recorded in the global comparison DAG before calling `Repair`.

Lemma 5.5 gives degree at most k and directed paths of at most $2H$ vertices during restoration. At each refinement these bounds and positivity allow Lemma 4.2 to be used. The chosen anchor is interior, and Lemma 5.2 preserves the invariant and positivity for the next step. Restoration terminates with the strict bounds. This completes the induction. Lemmas 3.1 and 4.2 ensure that every probability estimate and anchor selection terminates. The preparation and restoration lemmas bound the number of refinements in a call by $n + 2$, so the call terminates as well. The anchor estimates used at each step apply to arbitrary positive-volume node regions with the stated in-node DAG bounds. The proof of Lemma 3.1 in Section 6 applies to all such regions independently of the adversary's history. □

5.5 Why a long path requires deep anchors

Proof of Lemma 4.5. Every in-node arc on the selected path records an answered comparison. The path's labels are therefore pairwise comparable in the global comparison DAG, and remain so after the in-node arcs are deleted. By Lemma 5.2, two of these labels cannot be admitted to the same child of their parent in the adversary's tree. The procedure completes all $H = 2^k$ refinements in this parent, so its anchors occupy H distinct nodes of the implicit anchor tree.

The smallest possible sum of depths uses the $2^k - 1$ nodes at depths $0, \dots, k-1$ and one node

at depth k. Its value is

$$\sum_{m=0}^{k-1} m2^m + k = (k-2)2^k + k + 2 \geq H(k-2).$$

Every other choice of H distinct nodes has at least this sum of depths. □

6 Computing comparison probabilities

We now implement `Estimate` and prove Lemma 3.1. The calculation retains a bounded neighborhood of the queried items and computes the probability of the requested event exactly in that smaller region. We first show why this probability approximates the one in the full region, then give the finite calculation.

6.1 The region used by the estimator

Let G be the in-node DAG and K its feasible region. First compute the effective endpoints ℓ_z, r_z using all of G, in temporary storage. These endpoints preserve interval restrictions transmitted by paths that may extend beyond the neighborhood we will retain.

For a threshold query $X_x < c$, retain the vertices at undirected distance at most ρ from x. For a pair query $X_x < X_y$, retain the union of the two such neighborhoods. Keep every arc between retained vertices and copy their effective endpoints from G. Write V' for this set of vertices, and let

$$K' = \{z \in \mathbb{R}^{V'} : \ell_i < z_i < r_i \text{ for all } i \in V', \quad z_i < z_j \text{ for every retained arc } i \to j\}.$$

The region K' has positive volume: it contains the projection onto V' of every feasible assignment in K, including an open set. For the requested event E, the estimator returns

$$\widehat{p} = \frac{\mathrm{vol}(K' \cap E)}{\mathrm{vol}(K')}. \tag{26}$$

If a directed path in G already decides a pair query, or the threshold lies outside the queried effective interval, the answer is exactly zero or one and no volume calculation is needed.

The following theorem supplies the radius. Its dependence only on the stated parameters is what makes the exact calculation possible in polynomial time after the parameter choice in Section 7.

Theorem 6.1 (Local probability estimate)**.** *Let $d, h \geq 1$ be integers. Suppose G has degree at most d, its directed paths have at most h vertices, and $K \subseteq (0,1)^{V(G)}$ has positive volume. For every rational $R \geq 1$ and $0 < \varepsilon < 1/4$, there is a computable integer $\rho = \rho(d, h, R, \varepsilon)$ with the following properties.*

1. *If $W_x \geq 1/R$, the estimate in (26) for $X_x < c$ differs from its probability in K by at most ε.*
2. *If at least one of W_x, W_y is at least $1/R$, the same holds for $X_x < X_y$.*

The retained region has at most

$$b = 2(d+1)^{\rho+1} \tag{27}$$

coordinates. All its interval bounds are the effective bounds computed in the full graph.

6.2 Why distant coordinates have little effect

We put uniform assignments on K and K' in a common random experiment and bound the probability that they disagree on the query.

Proof of Theorem 6.1. Updating the two assignments. Choose a topological order of G and use its restriction to V' for the local region. One pass through this order will be called a *sweep*.

For an assignment uniform on either region, conditioning on the other coordinates leaves a uniform distribution for X_z on the interval between

$$L_z^X = \max(\{\ell_z\} \cup \{X_u : u \to z\}), \qquad R_z^X = \min(\{r_z\} \cup \{X_u : z \to u\}).$$

Thus replacing X_z by $(1 - U_z)L_z^X + U_z R_z^X$, with U_z uniform on $(0, 1)$, preserves the uniform distribution on the feasible region. This applies to each update in either graph.

Start X uniformly on K and Y uniformly on K'. At every retained vertex, use the same fresh uniform number for their two updates. Use independent fresh numbers at different vertices and sweeps. Both assignments remain uniform on their own regions throughout this joint experiment.

The decrease after one sweep. For $z \in V'$, write $\Delta_z = |X_z - Y_z|$. Superscripts old and new refer to the start of a sweep and to the value after a coordinate's update. Whenever all neighbors of z are retained, the two updates use the same interval bounds and give

$$\Delta_z^{\text{new}} \le (1 - U_z) \max_{u \to z} \Delta_u^{\text{new}} + U_z \max_{z \to u} \Delta_u^{\text{old}}, \tag{28}$$

where U_z is their shared random number and an empty maximum is zero. The predecessors have already been updated, while the successors retain their old values.

Let $B_h(z)$ denote the vertices within undirected distance h of z, and suppose this whole set is retained. Follow a predecessor attaining the first maximum in (28), and continue backwards until reaching a source. The resulting path has at most h vertices. All its vertices and their successors lie in $B_h(z)$, so every inequality used in this expansion is valid in both graphs. At the source, the lower-endpoint term in (28) is zero. It follows that

$$\Delta_z^{\text{new}} \le (1 - A_z^h) \max_{u \in B_h(z)} \Delta_u^{\text{old}}, \tag{29}$$

where A_z is the minimum of $1 - U_u$ over z and all its ancestors. Indeed, expansion along the selected path gives a factor $1 - \prod_u (1 - U_u)$, which is at most $1 - A_z^h$.

We next bound a high moment of the discrepancy. This lets the decrease in (29) outweigh the number of possible vertices in its maximum. Both $B_h(z)$ and the set of ancestors used to define A_z have at most M vertices, where we set

$$M = (d + 1)^{h+1}, \qquad p = (4M^2)^{h+1}. \tag{30}$$

Put $a_0 = 1/(4M^2)$. A union bound gives $\mathbb{P}(A_z < a_0) \le M a_0 = 1/(4M)$. On the complementary event,

$$(1 - A_z^h)^p \le (1 - a_0^h)^p \le \frac{1}{1 + p a_0^h} = \frac{1}{1 + 4M^2} \le \frac{1}{4M}.$$

Here $(1 - v)^{-p} \ge 1 + pv$ for $0 \le v < 1$ gives the middle inequality. Consequently $\mathbb{E}[(1 - A_z^h)^p] \le 1/(2M)$. The fresh numbers defining A_z are independent of all discrepancies at the start of the sweep. Raising (29) to the power p and bounding the maximum by a sum therefore gives

$$\mathbb{E}[(\Delta_z^{\text{new}})^p] \le \frac{1}{2M} \sum_{u \in B_h(z)} \mathbb{E}[(\Delta_u^{\text{old}})^p] \le \frac{1}{2} \max_{u \in B_h(z)} \mathbb{E}[(\Delta_u^{\text{old}})^p]. \tag{31}$$

Keeping the boundary far enough away. This estimate is used only where the needed neighbors are retained. Write $\Delta_z(s)$ for the discrepancy after s sweeps. If $B_{sh}(z) \subseteq V'$, induction on s gives

$$\mathbb{E}[\Delta_z(s)^p] \le 2^{-s}. \tag{32}$$

For $s = 0$, every discrepancy is at most one. For the induction step, apply (31); each $u \in B_h(z)$ has $B_{(s-1)h}(u) \subseteq V'$, so the induction hypothesis applies to it. Thus a radius ht suffices for t sweeps. At its outer boundary the argument uses only the initial discrepancy bound.

From coordinate errors to comparison errors. To turn this into the required probability estimate, put $C = M + 1$ and choose

$$\begin{aligned} &\delta = \frac{\varepsilon}{8CR}, && \lambda = \lceil \log(1/\delta) \rceil, && \mu = \lceil \log(4/\varepsilon) \rceil, \\ &t = p\lambda + \mu, && \rho = ht. \end{aligned} \tag{33}$$

For each queried coordinate, (32) and Markov's inequality imply

$$\mathbb{P}(\Delta_z(t) > \delta) \le 2^{-t}\delta^{-p} \le 2^{-\mu} \le \varepsilon/4.$$

For an incomparable pair, choose the queried coordinate x with $W_x \ge 1/R$. There are at most M other coordinates comparable with x. Corollary A.2 gives

$$\mathbb{P}(|X_x - X_y| \le 2\delta) \le 4CR\delta = \varepsilon/2.$$

If both queried discrepancies are at most δ, the two assignments can disagree on $X_x < X_y$ only when $|X_x - X_y| \le 2\delta$. The disagreement probability is at most $2(\varepsilon/4) + \varepsilon/2 = \varepsilon$. For a threshold, disagreement requires either $\Delta_x > \delta$ or $|X_x - c| \le \delta$, so its probability is at most $\varepsilon/4 + 2CR\delta = \varepsilon/2$.

The difference between the probabilities of an event is at most the probability that its two indicators disagree. Since X and Y remain uniform on K and K', these are precisely the probabilities being compared. Finally, a radius-ρ neighborhood has at most $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$ vertices, proving (27). All constants above are computable from the stated parameters. □

6.3 Calculating the local volumes exactly

The number of retained coordinates is bounded independently of n for fixed adversary parameters. We can therefore enumerate their possible orders and positions relative to the interval endpoints. This gives a particularly direct exact calculation of (26).

Suppose there are r retained coordinates. Sort their interval endpoints, together with $0, 1$ and the threshold c if present. Consecutive distinct endpoints divide $(0, 1)$ into at most $2r + 2$ open segments. Enumerate a segment for each coordinate and a permutation of all r coordinates. Keep a case exactly when the assigned segments lie inside the coordinates' allowed intervals, their segment indices are nondecreasing along the permutation, and that permutation respects all retained arcs. Each feasible assignment outside a set of volume zero occurs in exactly one such case.

If segment j has length Δ_j and contains r_j coordinates in the specified order, the case has volume

$$\prod_j \frac{\Delta_j^{r_j}}{r_j!}. \tag{34}$$

Sum these volumes over all kept cases to obtain $\mathrm{vol}(K')$, and over those satisfying the query to obtain $\mathrm{vol}(K' \cap E)$. A pair query is decided by the permutation; a threshold query is decided by the assigned segment. The denominator is positive, as established above.

Lemma 6.2 (Cost of the local calculation)**.** *For $r \le b$ coordinates numbered $1, \dots, r$, and dyadic endpoints and threshold with at most Λ fractional binary digits, the two volumes and their ratio can be computed in at most $C_0(b)(\Lambda+1)^2$ bit operations, for a computable function C_0. The exponent of $\Lambda+1$ is independent of b.*

Proof. There are at most $(2r+2)^r r!$ cases. Write each segment length as $\Delta_j = a_j/2^\Lambda$. Every case volume in (34) has the common denominator $2^{r\Lambda} r!$, because $\prod_j r_j!$ divides $r!$. Its integer numerator is

$$\frac{r!}{\prod_j r_j!} \prod_j a_j^{r_j}.$$

The cases have disjoint interiors and lie in the unit cube. Both summed numerators are therefore at most $2^{r\Lambda} r!$ and have $O(r\Lambda + r\log(r+1))$ bits. Their ratio is the requested probability. Sorting the endpoints, checking the cases, and forming their numerators uses a number of operations depending only on b. Ordinary integer arithmetic gives the stated bound. □

Proof of Lemma 3.1. The strict inequalities defining the feasible region make it open, so the existence of a feasible assignment gives positive volume. Computing the effective endpoints in temporary storage preserves the full feasible region by Lemma 5.1. Apply Theorem 6.1 with $d = k$, $h = 2H$, and $\varepsilon = \eta$. The exact local calculation above returns the probability to which that theorem applies, so it has the required accuracy. It uses a finite deterministic enumeration with exact rational arithmetic. □

The next section chooses the parameters as a function of n and bounds the running time over the entire interaction.

7 A uniform polynomial-time implementation

For each fixed degree threshold k, the preceding sections give a finite deterministic algorithm. Its local calculations depend on k, while its interval endpoints depend on the comparison history. We first bound the size of the maintained state, then choose k slowly enough that the local calculations also take polynomial time. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

7.1 Bounding the state throughout the interaction

The first test in Algorithm 2 answers any comparison already implied by the global comparison DAG. Such a call leaves the stored state unchanged. Every other call starts with an incomparable pair and makes that pair comparable permanently. Thus at most $\binom{n}{2}$ calls can change the state, however often the sorter repeats a comparison.

Lemmas 5.4 and 5.5 bound the number of refinements in one call by $n+2$. Consequently the total number P of refinements, including those in a call still in progress, satisfies

$$P \le (n+2)\binom{n}{2}. \tag{35}$$

Each refinement creates at most one node in the adversary's tree, so that tree has at most $P+1$ nodes. The global comparison DAG has n vertices and at most $\binom{n}{2}$ arcs. The transcript is the output stream; its order constraints are retained in this DAG.

We store an anchor by its path in the implicit anchor tree. A path of length m gives its interval and position by m halvings of $[0,1]$; the position has $m+1$ fractional binary digits. Anchor comparisons use these exact numerical positions.

For a wide item, Lemma 5.3 bounds both the number of search tests and the anchor precision by

$$K_0(k) = \lceil \log(4(B+1)R) \rceil + 2.$$

For a narrow item, suppose the current endpoints have at most Λ fractional digits. Its positive effective width is at least $2^{-\Lambda}$. The selected anchor-tree interval has length $2^{-m} > W_x/8$, so $m < \Lambda + 3$. The new anchor therefore has at most $\Lambda + 3$ fractional digits. Tightening takes maxima and minima of endpoints and adds no digits. After P refinements,

$$\Lambda \le K_0(k) + 3P. \tag{36}$$

The narrow walk makes at most $\Lambda + 2$ steps, using integers with $O(\Lambda)$ bits at each step.

7.2 Choosing the degree threshold

Lemma 6.2 bounds a local volume calculation by $C_0(b)(\Lambda+1)^2$ bit operations, where the number b of retained coordinates depends only on k. The function C_0 is computable from the finite enumeration in its proof. Finding the neighborhood and its effective endpoints uses graph operations on at most n vertices, with a fixed polynomial cost in n and Λ.

Choose a computable integer $C(k)$ that bounds $K_0(k)$ and all parameter-dependent factors in one local calculation, including $C_0(b)$. For powers of two $k = 64, 128, \ldots$, a fixed program writes the complete parameter tuple and $C(k)$ in binary. On input size n, simulate this program for n steps. Among the tuples it has completed, choose the largest k satisfying

$$C(k) \le n. \tag{37}$$

Use these parameters for the entire interaction. If no tuple qualifies, answer all comparisons according to the fixed label order.

The initialization is polynomial: the simulation writes at most n bits, and its completed tuples can be parsed and compared within a polynomial bound. For every fixed k, the tuple is written in some finite time $T(k)$. Once $n \ge \max\{T(k), C(k)\}$, it is available and affordable. Hence the chosen $k(n)$ tends to infinity.

For a selected tuple, (35)–(37) give $\Lambda = O(n^3)$ and a polynomial number of stored nodes. Every local volume calculation has a fixed polynomial bit cost. A comparison call makes at most $n + 2$ refinements, and each wide refinement uses at most $K_0(k) \le n$ estimates. Narrow refinements use the bounded walks described above. Routing queries, propagating bounds, and testing reachability take polynomial time in the stored state. Directed-path lengths are computed in reverse topological order; if $H > n$, no path can require repair. All tie rules can be implemented by scanning labels in order. Thus each comparison is answered within one fixed polynomial number of bit operations, independent of the chosen parameter and of the length of the preceding interaction.

Proof of Theorem 1.1. Initialization and every comparison call are deterministic and take polynomial time. For all sufficiently large n, a parameter tuple qualifies. Theorem 4.1 gives legality and forces

$$q \ge \frac{n \log n - 3n}{1 + (2 \log k(n) + 18)/k(n)} = (1 - o(1)) n \log n,$$

since $k(n) \to \infty$. The fixed-order rule for the remaining input sizes is also deterministic and legal. □

A Coordinate densities

We prove the density bounds used to choose anchors and to turn coordinate approximation into probability approximation. Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \neq x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \mathrm{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\mathrm{vol}_N K}, \qquad F_x(t) = \int_{-\infty}^{t} f_x(s)\, ds.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.1 (Coordinate density bounds)**.** *Suppose x has at most B other comparable labels and effective interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \leq \frac{B+1}{W}, \tag{38}$$

$$f_x(t) \geq \frac{4^{-B}}{W} \qquad (\ell + W/4 \leq t \leq r - W/4). \tag{39}$$

These bounds also hold after fixing any set of labels incomparable with x, for every resulting slice of positive volume.

Proof. We may fix any specified set of coordinates incomparable with x before starting the argument. Measure all volumes in the remaining free coordinates, and abbreviate the section volume by $S(t)$.

First we construct a map from the section at t into the section at a larger value s. Raising x may force its descendants to rise, but no other coordinates need move. We compress $[t, r]$ into $[s, r]$ and leave values above r unchanged. More precisely, for $\ell < t < s < r$, put $\alpha = (r-s)/(r-t)$ and apply

$$\phi(z) = \begin{cases} s + \alpha(z - t), & t \leq z \leq r, \\ z, & z \geq r \end{cases}$$

to x and all its descendants. Every descendant y has value greater than t. Its effective upper endpoint satisfies $r_y \geq r_x = r$, by the definition of effective intervals. A moved coordinate never decreases; if it starts below r, it stays below r, and if it starts at or above r, it stays fixed. Its lower and upper interval bounds are therefore preserved. The function ϕ is strictly increasing, so it preserves the inequalities imposed by arcs between moved coordinates. Every arc crossing the boundary of this set must enter it: an arc leaving a descendant would lead to another descendant. Only its head coordinate can move, and that coordinate never decreases, so its inequality also remains strict.

This injective map sends the section at t into the section at s. To compare volumes, partition the source section according to which moved coordinates lie below r. On each piece the map is affine and its Jacobian determinant in the free coordinates is α^j, where $j \leq B$: the coordinate x is fixed, and at most B other coordinates are moved. Since $0 < \alpha < 1$, this determinant is at least α^B. The images of distinct pieces are disjoint because the map is injective. Adding their volumes gives

$$S(s) \geq \left(\frac{r-s}{r-t}\right)^B S(t) \qquad (t < s). \tag{40}$$

For a smaller target value s, the construction is reversed: lower x and its ancestors, compress $[\ell, t]$ into $[\ell, s]$, and leave values below ℓ fixed. Thus, for $\ell < s < t < r$, use

$$\phi(z) = \begin{cases} z, & z \le \ell, \\ s + \dfrac{s-\ell}{t-\ell}(z-t), & \ell \le z \le t. \end{cases}$$

on x and its ancestors. The effective lower endpoint of every ancestor is at most ℓ. No moved coordinate rises, and one that starts at or above ℓ remains there. Thus all intervals are preserved. The map is strictly increasing on the moved set, so it preserves the inequalities imposed by arcs within that set. Every arc crossing its boundary leaves the set. Only its tail coordinate can move, and that coordinate never increases, so these inequalities also remain strict. Its determinant on each piece is at least $((s-\ell)/(t-\ell))^B$. Consequently

$$S(s) \ge \left(\frac{s-\ell}{t-\ell}\right)^B S(t) \qquad (s < t). \tag{41}$$

Neither construction moves a coordinate incomparable with x, so both remain valid after the conditioning at the start of the proof. A region of positive volume has some section of positive volume. The two maps carry that section to every $t \in (\ell, r)$ with a positive volume factor. Consequently, after fixing incomparable coordinates, we normalize the density of x over the same effective interval (ℓ, r) of width W.

Normalize S to obtain f_x. Integrating the two inequalities on their respective sides of t gives

$$1 \ge f_x(t)\left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^B ds + \int_t^r \left(\frac{r-s}{r-t}\right)^B ds\right) = \frac{W f_x(t)}{B+1}.$$

This proves the upper bound. For the lower bound, the density integrates to one over an interval of length W, so some $t_0 \in (\ell, r)$ has $f_x(t_0) \ge 1/W$. Fix $s \in [\ell + W/4, r - W/4]$. If $t_0 < s$, the ratio in (40) is at least $(r-s)/W \ge 1/4$; if $s < t_0$, the ratio in (41) is at least $(s-\ell)/W \ge 1/4$. If $s = t_0$, no map is needed. In every case $f_x(s) \ge 4^{-B}/W$. □

Corollary A.2 (A bound for small differences)**.** *If y is incomparable with x, or Y is a fixed number, then for every fixed interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \le \frac{(B+1)a}{W}, \tag{42}$$

where $Y = X_y$ in the first case.

Proof. In the first case, condition on X_y. Lemma A.1 bounds the conditional density of x by $(B+1)/W$. Integrating over the translate $I + X_y$ and then averaging over X_y proves the claim. For fixed Y, apply the unconditional density bound. □

To bound the density at an anchor selected by the search, we use log-concavity. A nonnegative function g is *log-concave* if

$$g((1-\theta)u + \theta v) \ge g(u)^{1-\theta} g(v)^\theta \qquad (0 < \theta < 1).$$

Prékopa's marginal theorem states that integrating out coordinates of a log-concave probability density preserves log-concavity [11, Theorem 8].

Lemma A.3 (Log-concavity of coordinate densities)**.** *Every coordinate density f, extended by zero outside its effective interval, is log-concave. In particular, for $u \le c \le v$,*

$$f(u)f(v) \le f(c)f(u+v-c). \tag{43}$$

Proof. The feasible region K is convex, so its uniform density, equal to $1/\operatorname{vol} K$ on K and zero elsewhere, is log-concave. Prékopa's theorem therefore applies to the coordinate density obtained by integrating out the other coordinates.

For the displayed inequality, the cases $c = u$, $c = v$, and $f(u)f(v) = 0$ are immediate. Otherwise put $\theta = (c-u)/(v-u)$. The points c and $u+v-c$ are the convex combinations of u, v with weights $(1-\theta, \theta)$ and $(\theta, 1-\theta)$, respectively. Multiplying the two log-concavity inequalities proves the claim. □

Lemma A.4 (Density from mass on both sides)**.** *For a coordinate density f, its cumulative distribution F, and $a \le c \le b$,*

$$(b-a)f(c) \ge \min\{F(c), 1-F(c)\}\big(F(b)-F(a)\big). \tag{44}$$

Proof. Lemma A.3 gives $f(u)f(v) \le f(c)f(u+v-c)$ for $u \le c \le v$. For fixed $v \ge c$, integration over $u \le c$ gives

$$F(c)f(v) \le f(c)\int_{-\infty}^{c} f(u+v-c)\,du \le f(c),$$

because the integral of a density over any interval is at most one. Integrating this inequality over $c \le v \le b$ yields

$$F(c)\big(F(b)-F(c)\big) \le (b-c)f(c).$$

Likewise, for fixed $u \le c$, integration over $v \ge c$ gives $f(u)(1-F(c)) \le f(c)$. Integrating over $a \le u \le c$ yields

$$\big(F(c)-F(a)\big)\big(1-F(c)\big) \le (c-a)f(c).$$

Add these inequalities and replace the two tail factors on the left by their minimum. □

B Parameter reference

Table 1 collects the parameters of the adversary. The last two columns distinguish where a parameter's role is introduced from where its value is chosen and explained. All choices are fixed before the first query. The constants used to bound the estimator's neighborhood are given in (30) and (33); its size is bounded by (27).

Table 1: Parameters and their roles. Section references lead to the explanations in the main text.

Symbol	Value	Role	Introduced	Choice
k	Power of two, $k \ge 64$	Refine a vertex when it has at least this many neighbors.	§3.2	§4
H	2^k	Number of vertices refined in a selected directed path.	§3.2	§4.2
B	$(k+1)^{2H+1}$	Bounds the number of other vertices comparable with one vertex in its in-node DAG during a repair.	§4.1	§4.1
R	2^{2kB}	Sets the width threshold: a coordinate is wide when $W_x \ge 1/R$.	§3.2	§5.3.3
η	$1/(100k)$	Maximum additive error in a probability estimate.	§3.2	§4.1

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.

[11] András Prékopa. *On logarithmic concave measures and functions.* Acta Scientiarum Mathematicarum 34, 335–343, 1973. Publisher's paper.